

Deep Learning Optimized on Jean Zay

Distribution – Data parallelism

IDRIS

BY NC SA

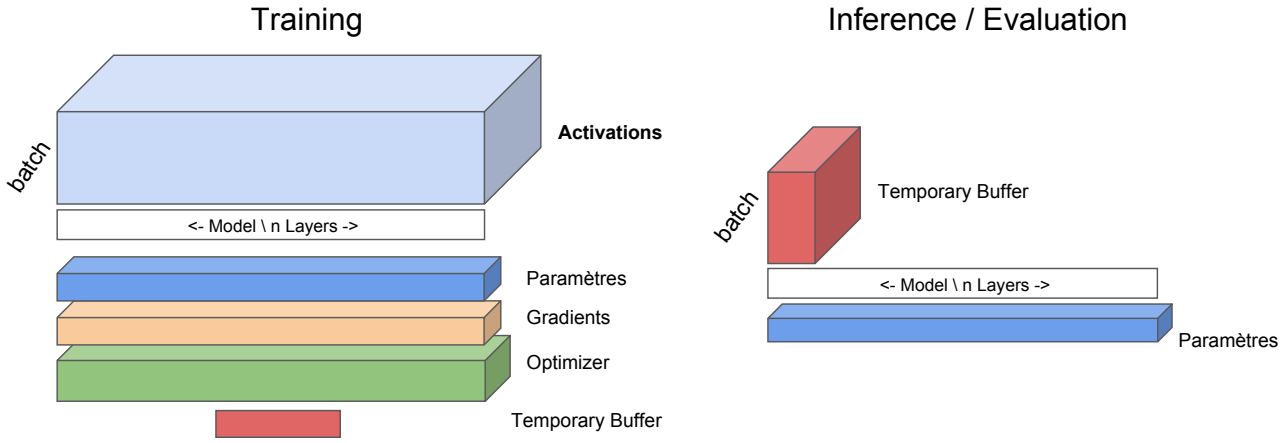
Distributed training: Data parallelism

Why distribute your training ◂

General knowledge about parallel computing ◂

Data parallelism to distribute your training ◂

Why distribute your training



When training on one GPU, one quickly reaches issues of scaling:

While increasing batch size speeds up and smooths training, it also translates to an increase in GPU memory footprint.

Using a bigger model can also bring some benefits, also at the cost of an increasing cost in GPU memory.

While techniques to mitigate the model's memory footprint exist, we will focus today on reducing the data footprint through the method called Distributed Data Parallel.

Distributed training: Data parallelism

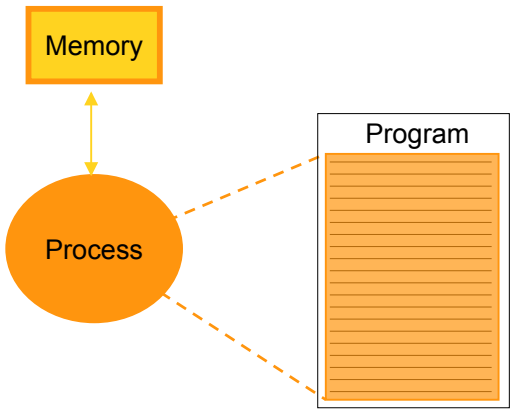
Why distribute your training ◂

General knowledge about parallel computing ◂

Data parallelism to distribute your training ◂

Let’s see for now the general principles of parallel computing.

Distribution: General knowledge about parallel computing

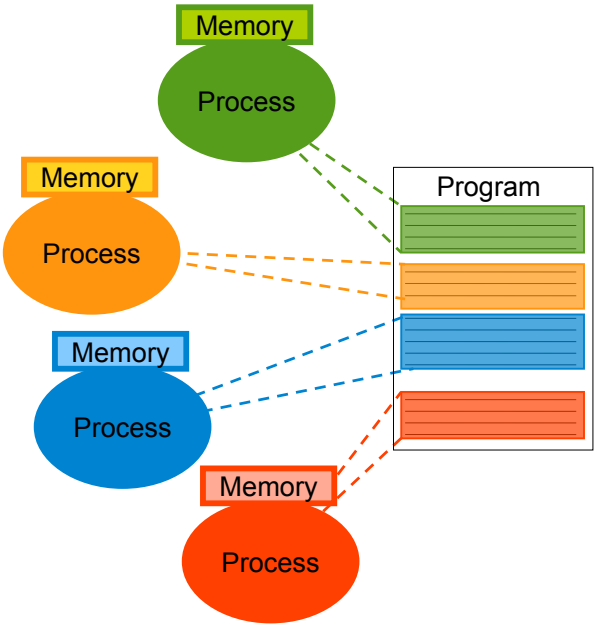


Sequential execution

- Only one process executes the program.
- The variables defined in the program are stored in the memory allocated to the process.
- One process executes the code on one physical compute unit (CPU core or GPU).

During a sequential (non-parallel) execution, a single process handles and has access to the entirety of the allocated memory (part of the system's RAM and/or VRAM) and executes the program on a single compute unit.

Distribution: General knowledge about parallel computing



Parallel execution with distributed memory

- Several processes execute the code at the same time (multiprocessing).
- The variables defined in the program are private, they are stored in the local memory allocated to each process.
- It is possible that the processes execute separate parts of the code.

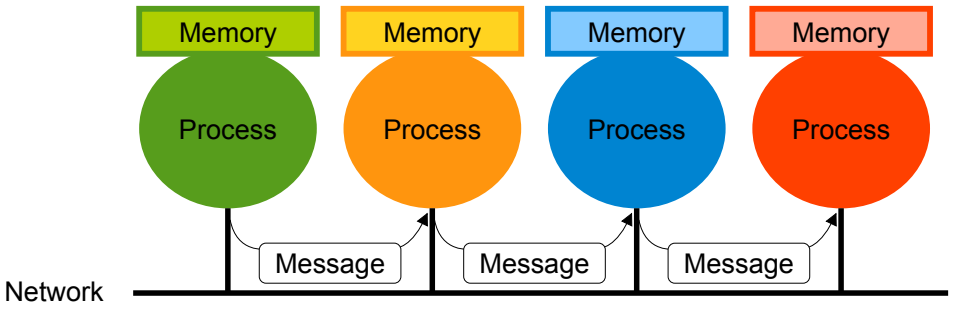
Conversely, during a parallel execution, multiple processes handle the execution at the same time, each having access only to the part of memory which is allocated to them.

NB: It's perfectly possible that the processes execute separate parts of the code.

Distribution: General knowledge about parallel computing

Parallel execution with distributed memory

- To share information, the processes can send each other **messages** through the interconnection network.
- These **communications** are managed by libraries such as **MPI** or **NCCL**.



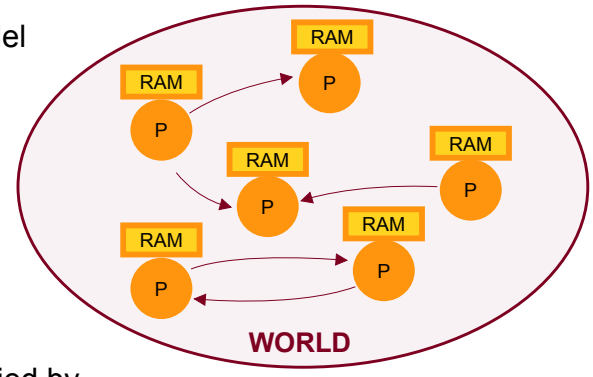
A single process handles and has access to its own memory only. However, during computation, one process will inevitably need results from one or several others, which calls for an **interconnection network** through which processes will **communicate**. Without any communications, we would essentially be running N of the same code. While spawning similar, parallel tasks which do not communicate can be a strategy (when doing a vast amount of simulations with different input parameters for example), when training a Neural Network this offers little benefits. Processes need to work collaboratively on a common task, hence the need for an efficient interconnection network.

These communications will be handled by libraries such as **MPI** (**Message Passing Interface**) or **NCCL** (**Nvidia Collective Communication Library**).

Distribution: General knowledge about parallel computing

In a parallel code based on the MPI or NCCL library,

- The set of processes exists in a common parallel **environment** initialized at the beginning of the execution.
- From the initialization to the destruction of this parallel environment, **all the processes** read and execute the program.
- During the initialization of the environment, a **communicator WORLD** is created to allow the set of processes to communicate with each other.
- The communicator has a **size** (the number of processes).
- Within a communicator, each process can be identified by its **rank**.



Whether in MPI or NCCL, communications are organized in what is called a **World**, whose size is the number of parallel processes. To identify these processes we allocate one or several numeric IDs called a **Global Rank**, though a second, local rank limited to the process' local node can be assigned.

Distribution: General knowledge about parallel computing

Example 1: Each process reads and executes the code lines

- Parallelized code using pytorch:

```
import torch.distributed as dist
dist.init_process_group()
size = dist.get_world_size()
print(f'The communicator size is {size}')
```

- Execution on 4 processes (**`#SBATCH --ntasks=4`**):

```
$ srun --ntasks=4 <...> python script.py
The communicator size is 4
The communicator size is 4
The communicator size is 4
The communicator size is 4
```


1st example in Pytorch:

We spawn 4 processes through the **parallel launcher srun**. All process run the same code, and thus output the same thing.

“srun” is Slurm’s parallel launcher, a program that handles the spawning of several processes on allocated Slurm ressources. Other spawners exist, such as mpirun or torchrun.

Distribution: General knowledge about parallel computing

Example 2: We identify the processes thanks to their ranks

- Parallelized code using pytorch:

```
import torch.distributed as dist
dist.init_process_group()
size = dist.get_world_size()
rank = dist.get_rank()
print(f'I am proc {rank} among {size}')
```

- Execution on 4 processes (**`#SBATCH --ntasks=4`**):

```
$ srun --ntasks=4 <...> python script.py
I am rank 1 among 4
I am rank 3 among 4
I am rank 2 among 4
I am rank 0 among 4
```


2nd exemple in Pytorch:

A process is identified through its rank. Note that, because they run in parallel, there is not reason for rank 0 to finish before rank 1.

The rank is a private, local variable created in each process' memory, but with a different value.

Distribution: General knowledge about parallel computing

Example 3: The processes can be assigned different tasks according to their ranks

- Parallelized code using pytorch:

```
import torch.distributed as dist
dist.init_process_group()
size = dist.get_world_size()
rank = dist.get_rank()
if rank%2==0:
    print(f'I am proc {rank}, my rank is even')
else:
    print(f'I am proc {rank}, my rank is odd')
```

- Execution on 4 processes (**`#SBATCH --ntasks=4`**):

```
$ srun --ntasks=4 <...> python script.py
I am proc 2, my rank is even
I am proc 0, my rank is even
I am proc 1, my rank is odd
I am proc 3, my rank is odd
```


3rd exemple in Pytorch:

You can change the instructions a process will execute depending on its rank.

Distribution: General knowledge about parallel computing

Example 4: Parallelization of a compute loop

- Initial state of the memory

↓Process 0↓

```
N = 4
a = [0, 1, 2, 3]
b = [4, 5, 6, 7]
c = [0, 0, 0, 0]
sum = 0
```

- Program

```
for i in range(N) :
    c[i] = a[i] + b[i]
    sum += c[i]
```

- Final state of the memory

```
N = 4
a = [0, 1, 2, 3]
b = [4, 5, 6, 7]
c = [4, 6, 8, 10]
sum = 28
```


4th exemple in Pytorch:

An actual computation of a result. This is the sequential version of a computation done with a “for” loop, which is typically what we want to distribute.

Distribution: General knowledge about parallel computing

Example 4: Parallelization of a compute loop

- Initial state of the memory

↓Process 0↓

```
N = 4
a = [0, 1, 2, 3]
b = [4, 5, 6, 7]
c = [0, 0, 0, 0]
sum = 0
```

↓Process 1↓

```
N = 4
a = [0, 1, 2, 3]
b = [4, 5, 6, 7]
c = [0, 0, 0, 0]
sum = 0
```

- Program

```
size = dist.get_world_size()
rank = dist.get_rank()

istart = rank * N / size
iend = (rank+1) * N / size

for i in range(istart,iend) :
    c[i] = a[i] + b[i]
    sum += c[i]
```

- Final state of the memory

```
N = 4
a = [0, 1, 2, 3]
b = [4, 5, 6, 7]
c = [4, 6, 0, 0]
sum = 10
```

```
N = 4
a = [0, 1, 2, 3]
b = [4, 5, 6, 7]
c = [0, 0, 8, 10]
sum = 18
```


4th exemple in Pytorch:

An actual computation of a result, parallel version. We cut the domain of computation in two by distributing the iterations between processes.

Distribution: General knowledge about parallel computing

Example 4: Parallelization of a compute loop

- Initial state of the memory

↓Process 0↓

```
N = 4
a = [0, 1, 2, 3]
b = [4, 5, 6, 7]
c = [0, 0, 0, 0]
sum = 0
```

↓Process 1↓

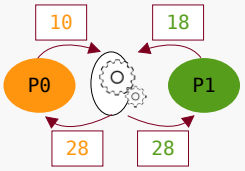
```
N = 4
a = [0, 1, 2, 3]
b = [4, 5, 6, 7]
c = [0, 0, 0, 0]
sum = 0
```

- Program

```
size = dist.get_world_size()
rank = dist.get_rank()

istart = rank * N / size
iend = (rank+1) * N / size

for i in range(istart,iend) :
    c[i] = a[i] + b[i]
    sum += c[i]
sum = dist.all_reduce(sum,op=dist.reduceOp.SUM)
```



- Final state of the memory

```
N = 4
a = [0, 1, 2, 3]
b = [4, 5, 6, 7]
c = [4, 6, 0, 0]
sum = 28
```

```
N = 4
a = [0, 1, 2, 3]
b = [4, 5, 6, 7]
c = [0, 0, 8, 10]
sum = 28
```

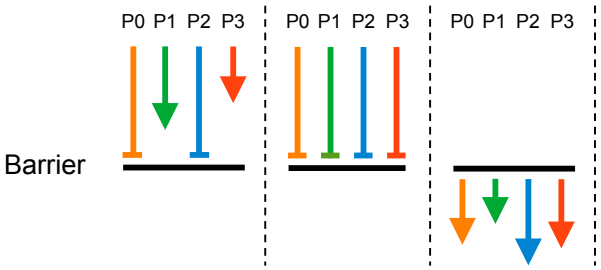

4th exemple in Pytorch:

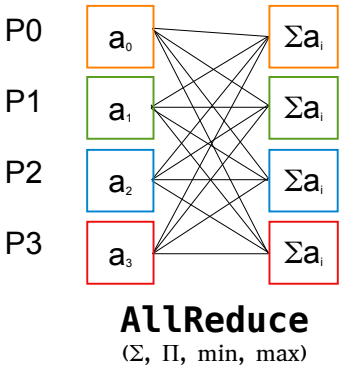
An actual computation of a result, parallel version. We cut the domain of computation in two and distribute each half to a process.

We now need both process to communicate and aggregate their respective results (here with a SUM operation).

Distribution: General knowledge about parallel computing

- Inter-process communication `AllReduce()`
  - Collective communication
  - Synchronization barrier







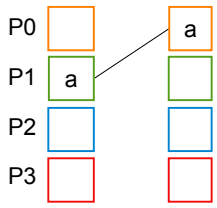
costly communication

The **AllReduce** operation is one of the main collective communication operations.

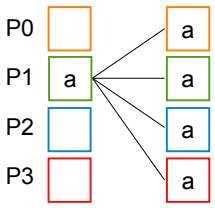
It naturally incorporates a synchronization barrier, that is to say, a point in the code that all processes must reach before continuing. This barrier is unavoidable in AllReduce, as we need to ensure that the intermediate results have been fully calculated before performing the reduction, and we need the reduced result to continue with the rest of the code (the backward propagation of a neural network for example).

Distribution: General knowledge about parallel computing

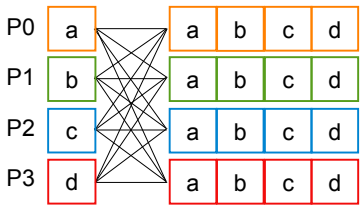
- NCCL communications



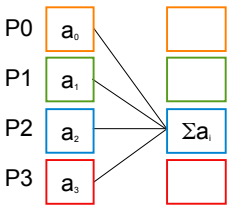
Send/Receive



Broadcast

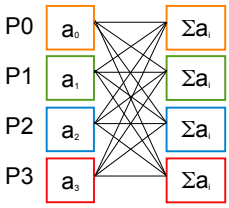


AllGather



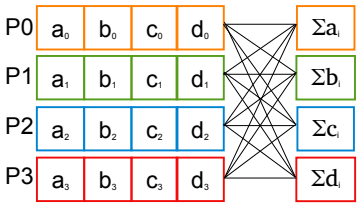
Reduce

(Σ, Π, min, max)



AllReduce

(Σ, Π, min, max)



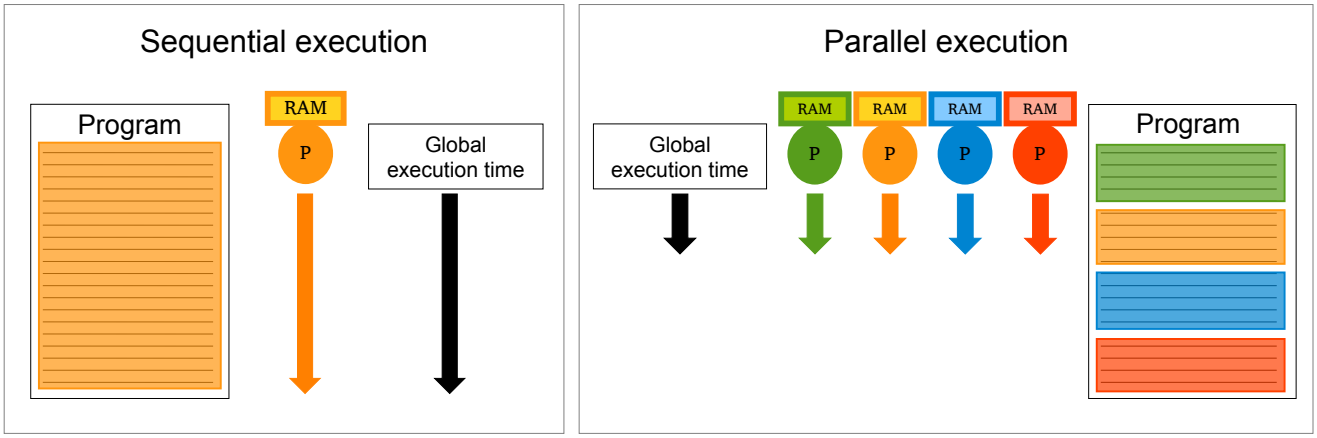
ReduceScatter

(Σ, Π, min, max)

The other NCCL communication methods are as follows:

- Send/Receive: One process sends data to another
- Broadcast: One process sends data to all the others
- AllGather: Each process holds a portion of the data and retrieves all the other portions
- Reduce: Each process holds a version of the data. One process retrieves them all and then applies a reduction operation to them.
- AllReduce: Each process holds a version of the data, and they all collect all versions and apply the same reduction operation to them.
- ReduceScatter: Each process holds a version of the data; it collects a specific part of each version and applies a reduction operation to them.

Distribution: General knowledge about parallel computing



- Global execution time: $T(Nprocs) = \frac{T_{parall}(1proc)}{N} + T_{seq} + T_{comm}$

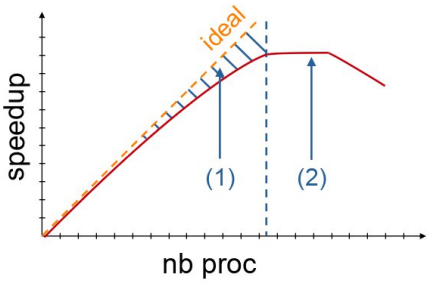
In theory, correctly parallelising a computation allows the total execution time to be divided by the number of parallel processes.

In practice, this execution time is always limited by a composite asymptote consisting, firstly, of the non-parallelisable sequential computation time – such as the initialisation of parallel processes and the communication context – and, secondly, of communication overhead, which, incidentally, increases as the number of processes rises, to the point where it can even cause performance to decline again.

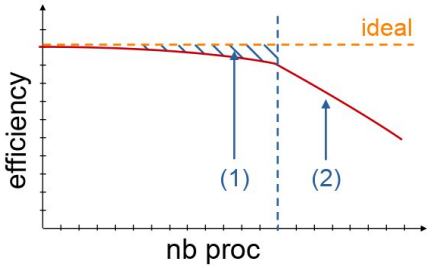
Distribution: General knowledge about parallel computing

Scalability study: $\frac{T(1)}{T(N)}$ (speedup or efficiency)

- **Strong scaling** (problem size is constant)
  - T(1) = time to complete a serial task
  - T(N) = time to complete the same parallel task with N processes



- **Weak scaling** (problem size is proportional to the number of processes)
  - T(1) = time to complete a single work unit
  - T(N) = time to complete N of the same work unit with N processing elements



(1) impact of the sequential parts of the code
(2) cost of the inter-process communications

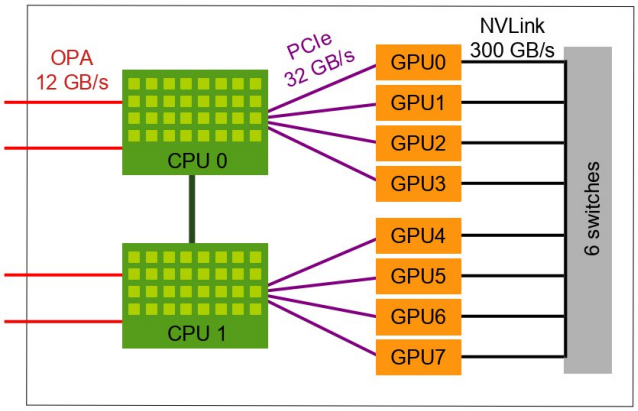
Parallelization can thus allow for two things:

First, decrease computation time for a fixed-size problem. This is called “strong scaling” and what we have discussed already. As we increase the number of processes, the return on investment in terms of speedup will start to decline, mainly because communications get more numerous, but also because the amount of computations done by each processing unit becomes ludicrous.

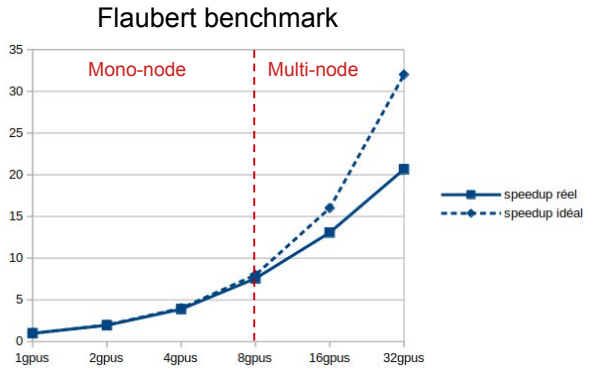
Second, we can use parallel processing to increase the size of our problem, usually multiply it by the same factor as our number of processes. While in theory we can accomplish a problem N times bigger with N times more processes, again, in practice, the efficiency (meaning the gain in speedup between two increase) will decline because of communications.

Distribution: General knowledge about parallel computing

- Bandwidths of the interconnection networks on Jean Zay:



Node 8 × A100 80GB



Strong scaling

On a machine like the Jean-Zay supercomputer, the slow-down caused by communications becomes more noticeable when reaching the multi-nodes threshold. This is due to the nodes interconnection bandwidth being lower than the GPU's.

The figure on the right shows that going from 8 to 16 A100 GPUs (meaning from 1 to 2 nodes), the relative speedup is inferior to the expected x2.

Distributed training

Why distribute your training ◂

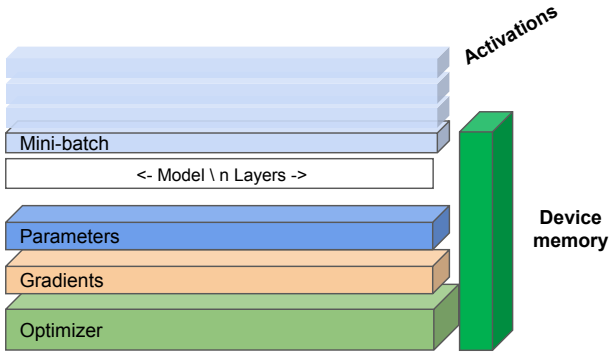
General knowledge about parallel computing ◂

Data parallelism to distribute your training ◂

Distribution: Data parallelism

Data parallelism

- Mini-batches distributed on different devices
- Model small enough to be contained on one device in memory
- Training time speedup
- Effectively increased batch size
- Causes large batches (consequences on training quality)



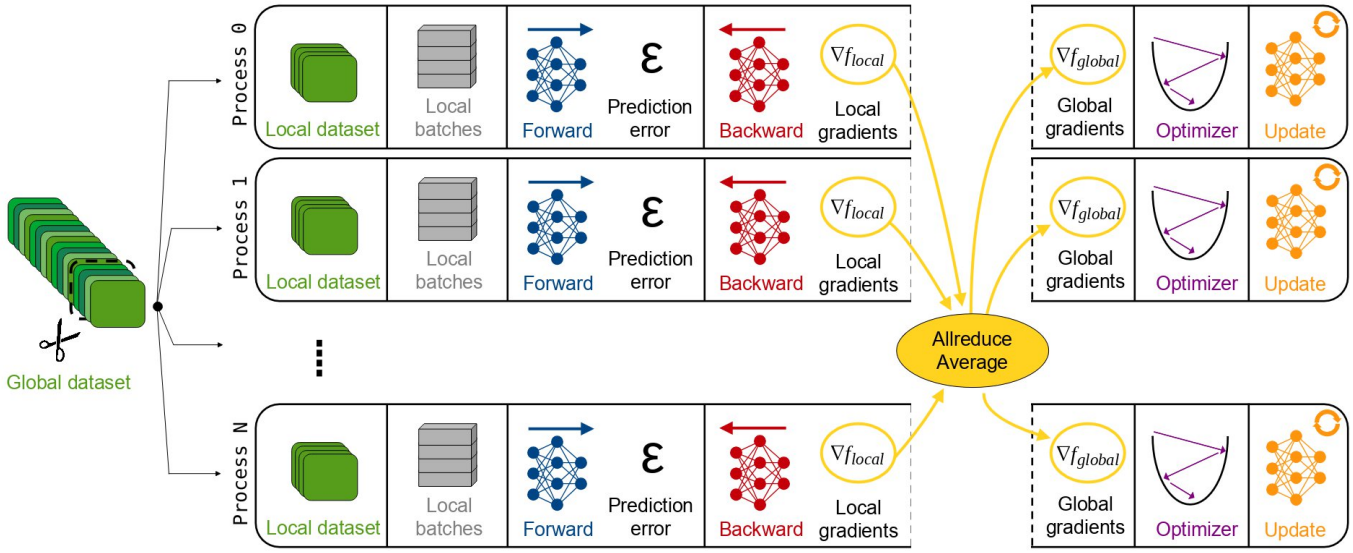
Why use Data Parallelism ?

Data Parallelism (or DP) is a technique used to speed-up training by cloning a small model – relatively-speaking – across multiple devices and distributing a different part of the data on each device.

On a one-device training, the maximum batch size is limited by the device memory. Once that ceiling is reached the next step is to add another device and distribute a batch to it, effectively doubling the batch size, meaning more data seen per epoch and thus a faster (and smoother) training.

The model stays identical between all devices throughout training, but as different parts of data are seen, the resulting loss and gradients will differ.

Distribution: Data parallelism



Because local gradients differ, but we wish for each version of the model to remain identical, the Data Parallelism pipeline introduces an Averaging **AllReduce** after gradient computation.

The whole pipeline is now the following:

– First the global dataset is split between all processes to then create **local mini-batches.**
– Then, each process does its own forward and backward to compute local gradients.
– As Data Parallelism is based on duplicating the same model, before the update is done we average all local gradients and distribute the resulting global gradients to all devices.

Each device is then ready to perform the weights update.

This results in an identical model between all devices, and an effective batch size equal to the local batch size multiplied by the number of devices.

Distribution: Data parallelism

Implementation of the data parallelism

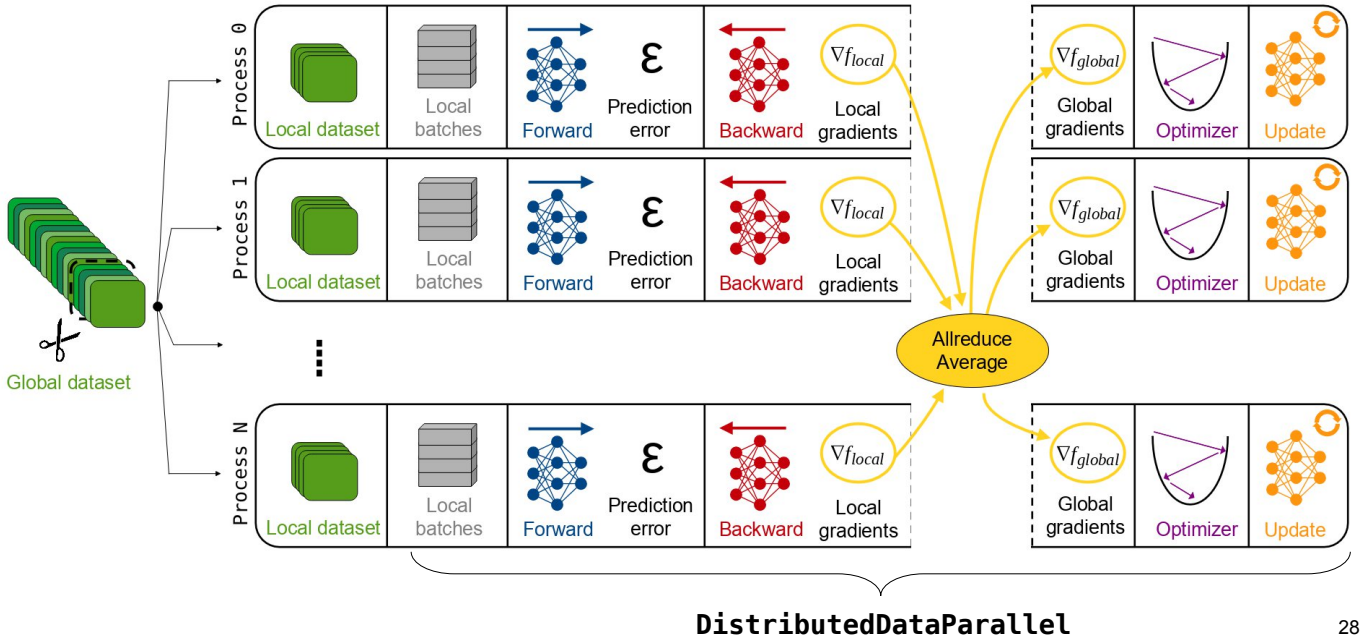
- PyTorch → **`DistributedDataParallel`** (integrated solution)

In PyTorch this is implemented by the DistributedDataParallel class, which wraps around an already-existing model to handle the distribution and parallel execution.

In TensorFlow this is done through the MultiWorkerMirroredStrategy interface.

This course will only focus on the PyTorch implementation.

Distribution: Data parallelism



The DistributedDataParallel class handles the forward, backward, gradient reducing and weight update steps.

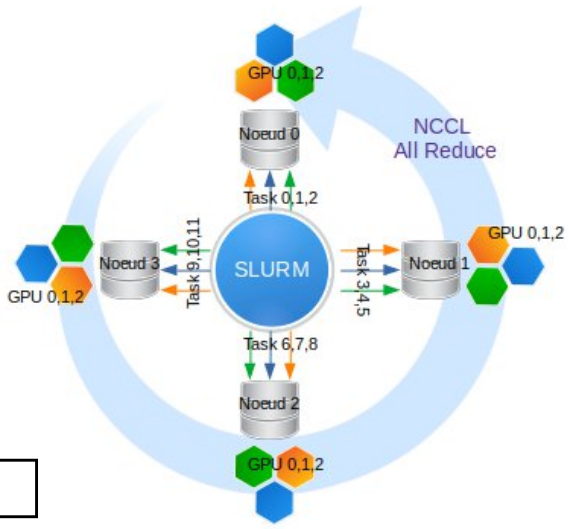
The distribution of dataloading is implemented separately through another PyTorch class, the DistributedSampler (which we will see later).

Distribution: Data parallelism

Execution of the parallel code → Slurm environment

- Distribution example on : 4 nodes
  3 GPUs per node

```
## Slurm script
#SBATCH --nodes=4             # nb nodes
#SBATCH --ntasks=12           # nb proc
#SBATCH --ntasks-per-node=3   # nb proc / node
#SBATCH --gres=gpu:3          # nb GPUs / node
srun python script.py
```



Each GPU must be bound to one process.

This is what a parallel environment looks like with 4 nodes and 3 GPUs per node.

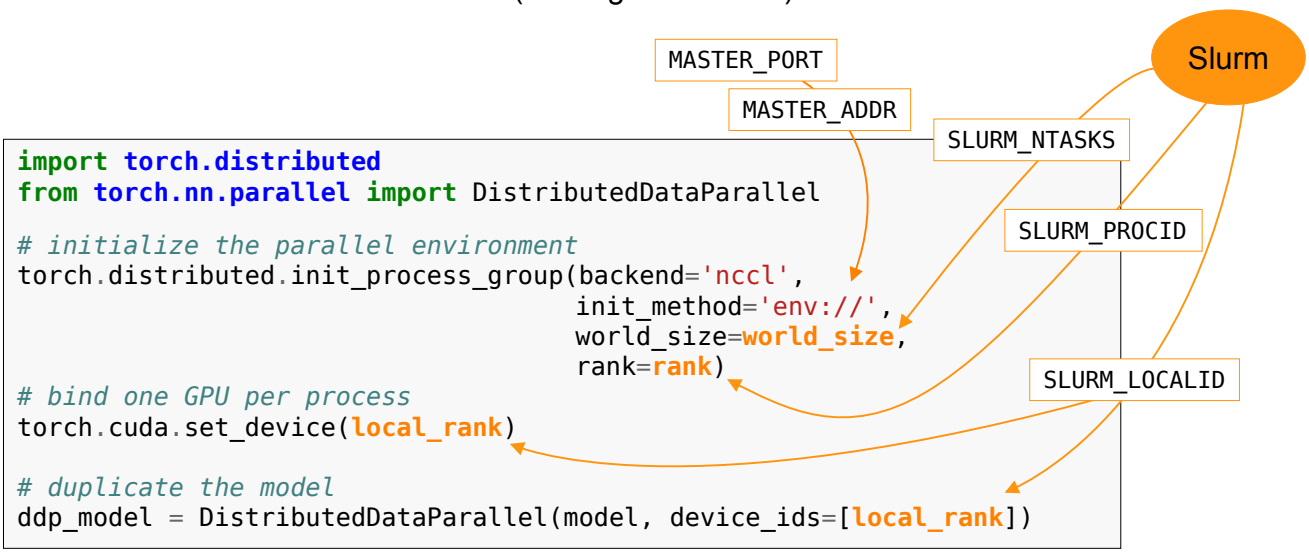
The slurm script should specify the number of nodes and GPU, and either the “--ntasks” option (total number of tasks across all nodes) or the “--ntasks-per-node” option.

(Reminder: “sbatch” does not spawn tasks. The “srun” command will handle the spawning and distribute the python command across all processes.)

In other words, every process runs the same python script. As we have seen, it is possible to change the code’s behaviour based on a process’ rank, and thus handle the parallelization from inside the code.

Distribution: Data parallelism

- **`DistributedDataParallel`** (training distribution)



To spawn a parallel environment and apply the DDP strategy (distributing a model), PyTorch needs to be provided with the necessary information:

“Backend” is the communication backend, most likely NCCL.

“init_method” receives the IP address of the machine hosting device of rank 0, as well as a free port on that machine. Using “env://” tells PyTorch to fetch this info through the environment variables MASTER_ADDR and MASTER_PORT.

“world_size” is the total number of processes (across all nodes)

“rank” is the *global_rank* of the current process.

It is then necessary to bind a GPU to the current process. This is done this time using the *local_rank*.

The local_rank is also used when creating the DDP model in the DistributedDataParallel constructor. It corresponds to the node-wise rank of the GPU the process is bound to. (As seen on the previous slide, all nodes have GPUs 0,1, and 2. This is their local ranks).

This is the interface between the ressource allocator (Slurm), its parallel spawner (surn) and the training framework (torch.distributed).

Distribution: Data parallelism

- **`DistributedDataParallel`** (training distribution)
  - *idr_torch.py* script from IDRIS

```python
# idr_torch.py
import os
import hostlist

# get SLURM variables
size = int(os.environ['SLURM_NTASKS'])
rank = int(os.environ['SLURM_PROCID'])
local_rank = int(os.environ['SLURM_LOCALID'])
cpus_per_task = int(os.environ['SLURM_CPUS_PER_TASK'])

# get node list from slurm
hostnames = hostlist.expand_hostlist(os.environ['SLURM_JOB_NODELIST'])

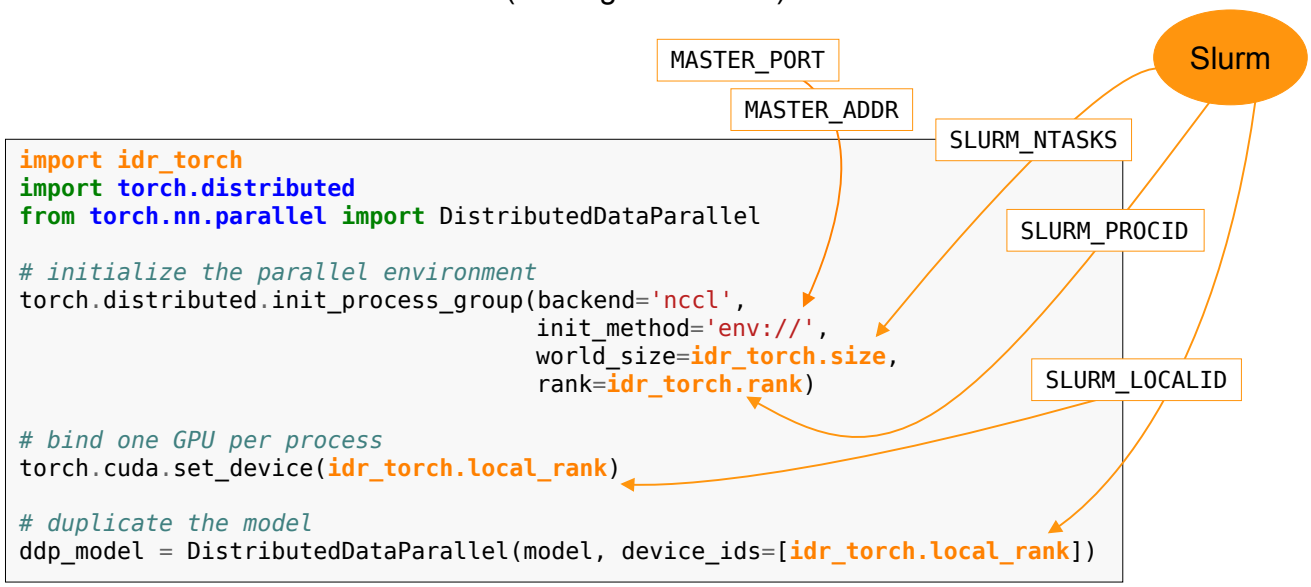
# get IDs of reserved GPU
gpu_ids = os.environ['SLURM_STEP_GPUS'].split(",")

# define MASTER_ADD & MASTER_PORT
os.environ['MASTER_ADDR'] = hostnames[0]
os.environ['MASTER_PORT'] = str(12345 + int(min(gpu_ids)))
```


In order to ease the implementation of parallel training, Jean Zay users have access to the idr_torch library, which serves as a wrapper to recover all these environment variables through an intuitive interface.

Distribution: Data parallelism

- **`DistributedDataParallel`** (training distribution)



In order to ease the implementation of parallel training, Jean Zay users have access to the idr_torch library, which serves as a wrapper to recover all these environment variables through an intuitive interface.

Distribution: Data parallelism

- What about `torchrun`? → Possible but cumbersome.

slurm.sh

```
[…]
#SBATCH --ntasks-per-node=1
[…]
GPUS_PER_NODE=8
MASTER_ADDR=$(scontrol show hostnames $SLURM_JOB_NODELIST | head -n 1)
MASTER_PORT=15000
CMD="train.py --arg1 1 --arg2 2"
export LAUNCHER="torchrun --nproc_per_node $GPUS_PER_NODE \
                          --nnodes $SLURM_NNODES \
                          --rdzv_backend c10d \
                          --rdzv_endpoint $MASTER_ADDR:$MASTER_PORT"
srun bash -c "$LAUNCHER --node_rank \$SLURM_PROCID $CMD"
```

train.py

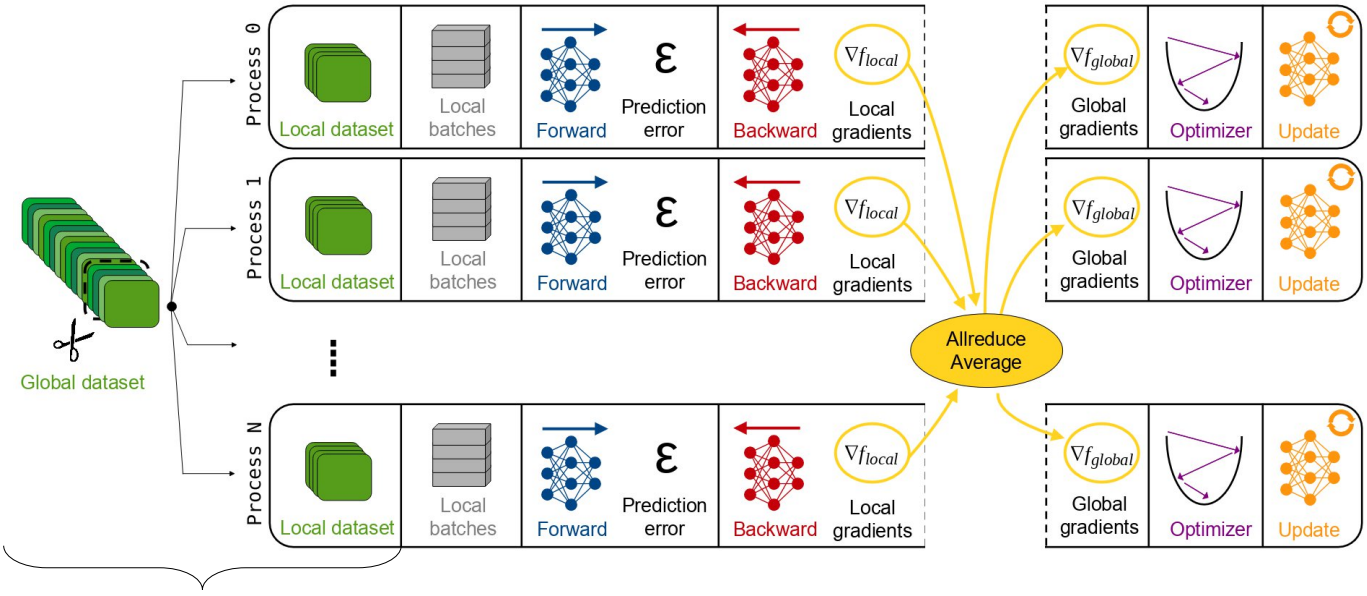
```
[…]
WORLD_RANK = int(os.environ['RANK'])
LOCAL_RANK = int(os.environ['LOCAL_RANK'])
WORLD_SIZE = int(os.environ['WORLD_SIZE'])
```


From the pytorch documentation:

“torch.distributed.run is a module that spawns up multiple distributed training processes on each of the training nodes. torchrun is a python console script to the main module torch.distributed.run declared in the entry_points configuration in setup.py. It is equivalent to invoking python -m torch.distributed.run.”

The most convenient way to launch parallel processes on Jean Zay is through srun. While it’s technically possible to use both torchrun and srun, this method is both cumbersome and redundant. srun automatically spawns the correct number of processes based on allocated resources, while torchrun requires setting multiple variables manually, as well as for communications.

Distribution: Data parallelism



If the training is handled by the DistributedDataParallel class, everything that has to do with “distributing” the dataloading is done through the DistributedSampler class.

This DistributedSampler is needed because the job of the dataloader is to prepare batches according to the sampler, but sampling data for distributed training requires a dedicated sampler object aware of the parallel environment.

Distribution: Data parallelism

- **`DistributedSampler`** (distributing the input data)

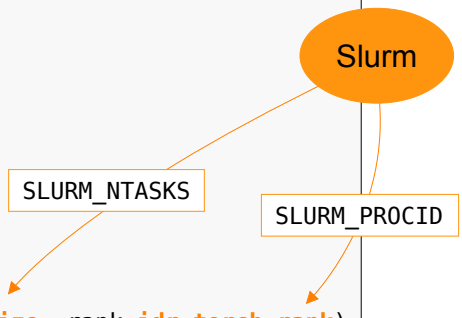
```python
import idr_torch
import torch.distributed
from torch.nn.parallel import DistributedDataParallel
from torch.utils.data.distributed import DistributedSampler

# initialize the parallel environmenet
[...]

# bind one GPU per process
[...]

# duplicate the model
[...]

# distribute the input data
data_sampler = DistributedSampler(dataset, shuffle=True,
                                  num_replicas=idr_torch.size, rank=idr_torch.rank)
```



The shuffling step is assigned to the Sampler.

As with other distributed objects, the distributed sampler requires knowledge about the parallel environment: the number of processes and the rank of the local process.

Data shuffling should be handled by the DistributedSampler instead of the dataloader to ensure the dataset is shuffled between each epoch across all processes and not just on each process' side.

Distribution: Data parallelism

- **`DistributedSampler`** (distributing the input data)

  - Parallel execution on 4 processes using the `DistributedSampler`:

```
dataset = [0, 1, ..., 99]
batch_size_per_gpu = 5
ntasks = 4
batch_size = 20
```

```
$ srun --ntasks=4 <...> python script.py
Rank 0: Batch 0 = tensor([ 0,  4,  8, 12, 16])
Rank 1: Batch 0 = tensor([ 1,  5,  9, 13, 17])
Rank 2: Batch 0 = tensor([ 2,  6, 10, 14, 18])
Rank 3: Batch 0 = tensor([ 3,  7, 11, 15, 19])
```


Example:

DistributedSampler without shuffling: Data is distributed in a regular manner between each device, but no shuffling is done device-side.

Distribution: Data parallelism

- **`DistributedSampler`** (distributing the input data) **+ shuffling**
  - **The index shuffling is performed by each GPU from a common seed.**

  - Parallel execution on 4 processes using the `DistributedSampler`:

```
dataset = [0, 1, ..., 99]
batch_size_per_gpu = 5
ntasks = 4
batch_size = 20
```

```
$ srun --ntasks=4 <...> python script.py
Rank 0: Batch 0 = tensor([46, 36, 80, 17, 23])
Rank 1: Batch 0 = tensor([16, 64, 97, 12, 59])
Rank 2: Batch 0 = tensor([91, 18, 49, 24,  4])
Rank 3: Batch 0 = tensor([33, 73, 37, 81, 63])
```


Example:

DistributedSampler with shuffling: Data is distributed in a regular manner between each device, and shuffling is done by each device with a common seed.

Distribution: Data parallelism

- **`DistributedSampler`** (distributing the input data) **+ shuffling**
  - **The index shuffling is performed by each GPU from a common seed.**

```
for epoch in range(1,30):

            for i, batch in enumerate(dataloader):
                        ...
```



```
>>> Epoch 1
Rank 0: Batch 0 = tensor([46, 36, 80, 17, 23])
Rank 1: Batch 0 = tensor([16, 64, 97, 12, 59])
Rank 2: Batch 0 = tensor([91, 18, 49, 24,  4])
Rank 3: Batch 0 = tensor([33, 73, 37, 81, 63])
>>> Epoch 2
Rank 0: Batch 0 = tensor([46, 36, 80, 17, 23])
Rank 1: Batch 0 = tensor([16, 64, 97, 12, 59])
Rank 2: Batch 0 = tensor([91, 18, 49, 24,  4])
Rank 3: Batch 0 = tensor([33, 73, 37, 81, 63])
```

```
for epoch in range(1,30):
            data_sampler.set_epoch(epoch)
            for i, batch in enumerate(dataloader):
                        ...
```



```
>>> Epoch 1
Rank 0: Batch 0 = tensor([46, 36, 80, 17, 23])
Rank 1: Batch 0 = tensor([16, 64, 97, 12, 59])
Rank 2: Batch 0 = tensor([91, 18, 49, 24,  4])
Rank 3: Batch 0 = tensor([33, 73, 37, 81, 63])
>>> Epoch 2
Rank 0: Batch 0 = tensor([49, 91,  8, 76, 48])
Rank 1: Batch 0 = tensor([98, 50, 21, 15, 22])
Rank 2: Batch 0 = tensor([ 2, 11, 71, 92, 75])
Rank 3: Batch 0 = tensor([82,  9, 74, 39, 53])
```


Important: Calling the set_epoch() method on the DistributedSampler at the beginning of each epoch is necessary to update the random seed and make shuffling work properly across multiple epochs. Otherwise, the same ordering will be used in each epoch.

Distribution: Data parallelism

- Custom Sampler (inspired by `DistributedSampler`)

```
class MyCustomDistributedSampler(Sampler):

          def __init__(self,dataset,world_size,rank):
                    self.datalen = len(dataset)
                    self.world_size = world_size
                    self.rank = rank

          def __len__(self):
                    return self.datalen

          def __iter__(self):
                    indices = list(range(self.datalen))
                    # shuffle or not shuffle
                    indices = indices[self.rank:self.datalen:self.world_size]
                    return iter(indices)


sampler = MyCustomDistributedSampler(dataset,idr_torch.size,idr_torch.rank)
```

```
$ srun --ntasks=4 <...> script.py
Rank 0: Batch 0 = tensor([ 0,  4,  8, 12, 16])
Rank 1: Batch 0 = tensor([ 1,  5,  9, 13, 17])
Rank 2: Batch 0 = tensor([ 2,  6, 10, 14, 18])
Rank 3: Batch 0 = tensor([ 3,  7, 11, 15, 19])
```

TP_PI: Parallel computation of PI using torch.distributed





- Go into the directory `tp_pi/`
- Follow instructions in the notebook **`DLO-JZ_Compute_pi.ipynb`**
- Parallelize the code `compute_pi.py` using `torch.distributed`
- Compute π on 4 GPUs

TP2_2: Implement data parallelism in dlojz.py





- Follow instructions in the notebook **`DLO-JZ_Jour2.ipynb`**
- Implement data parallelism in the script `dlojz.py`
- Measure the gain in time when using 4 GPUs